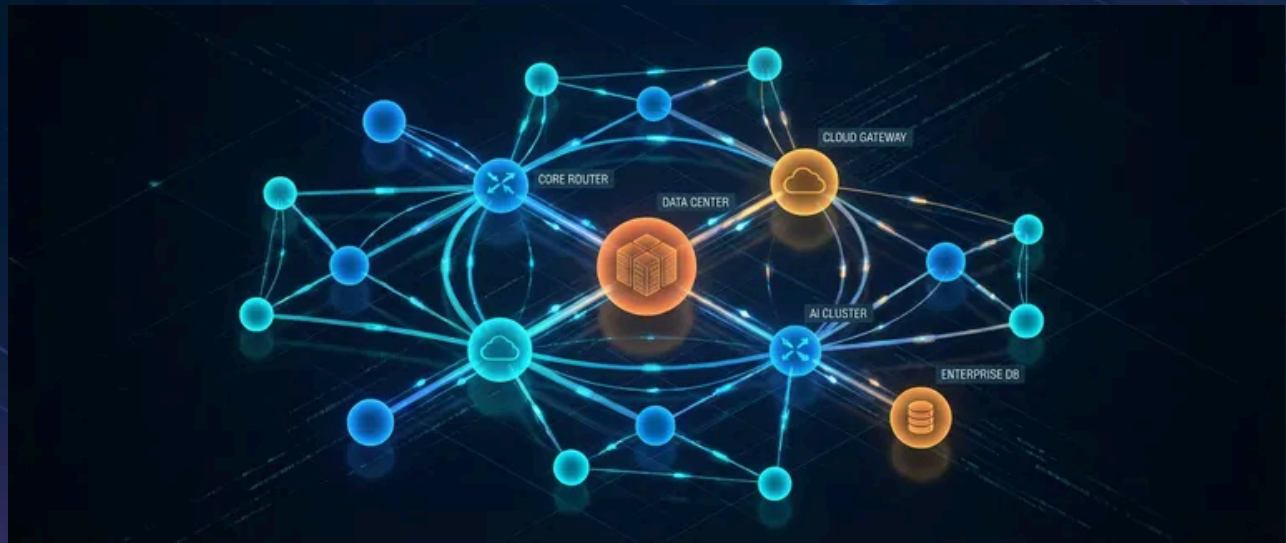


Service Catalog Schema: Metadata That Gets Used



Published on June 19, 2022



Webstack
Builders

Table of Contents

The Service Catalog Problem	3
Catalog Failure Modes	3
What Makes Catalogs Actually Useful	4
Catalog vs CMDB vs Service Mesh	5
Schema Design Principles	6
Core Entity Model	7
Required vs Optional Fields	8
Extension Points and Custom Metadata	10
Ownership Model Design	11
Ownership Hierarchy	12
Ownership Transfer Workflows	13
Orphan Detection and Resolution	15
Dependency Tracking	16
Dependency Types and Metadata	16
Automated Dependency Discovery	18
Dependency Graph Queries	20
Automation and Enforcement	21
Catalog-as-Code Pattern	21
CI/CD Integration	22
Automated Freshness Enforcement	24
Drift Detection	25
Catalog API and Integrations	26
API Design for Catalog Consumers	26
Integration Patterns	28
Measuring Catalog Health	32
Catalog Coverage Metrics	32
Alerting on Catalog Quality	35
Conclusion	37



Service catalogs follow a depressingly predictable arc. Leadership announces a new catalog initiative, teams dutifully enter their services, and six months later nobody trusts the data. The catalog becomes that thing you're supposed to update but don't, because checking Slack is faster than hoping the ownership field is current.

The failure isn't discipline or tooling – it's schema design and enforcement. A catalog that relies on humans remembering to update it will decay. One that validates data at deploy time, discovers dependencies from runtime traffic, and alerts on drift has a chance of staying accurate. The difference between a useful catalog and an expensive spreadsheet comes down to whether you treat metadata quality as a system property you measure and enforce, or a cultural goal you hope teams will adopt.

The Service Catalog Problem

Every service catalog I've inherited has been partially abandoned. The enthusiastic launch gave way to gradual neglect, until what should have been an asset turned into a liability. Understanding *why* catalogs fail is the first step toward building one that doesn't.

Catalog Failure Modes

The decay follows a predictable timeline. Month one sees high coverage and enthusiasm – everyone's entering their services because the initiative has leadership attention. By month three, new services are being deployed without catalog entries because it's not in the critical path. Month six brings the ownership rot: reorgs happen, people leave, but the catalog still points to teams that no longer exist. By year one, engineers have learned they can't trust the data, so they ask in Slack instead. Year two? The catalog is a punchline in onboarding jokes.

#	Time	Coverage	Accuracy	Team trust
1	Launch	90%	95%	85%
2	Month 3	85%	80%	70%
3	Month 6	78%	70%	50%
4	Year 1	65%	50%	25%



#	Time	Coverage	Accuracy	Team trust
5	Year 2	45%	30%	10%

Catalog trust decay over time.

Here’s the thing that makes this worse: a catalog with 80% accurate data is more dangerous than no catalog at all. It gives you false confidence. You page the listed owner at 3 AM, confident you’ve got the right team, and waste twenty minutes before discovering they handed off the service six months ago.

Failure Mode	Symptom	Root Cause
Stale ownership	Paging wrong team during incidents	No ownership update workflow
Missing services	Services in prod not in catalog	Catalog not in deployment path
Wrong dependencies	Dependency graph incomplete	Manual entry, no validation
Outdated metadata	Descriptions don't match reality	No freshness enforcement
Orphaned entries	Deleted services still listed	No lifecycle management

Common catalog failure modes and their root causes.

What Makes Catalogs Actually Useful

Before designing a schema, you need to know what questions the catalog should answer. I’ve found that catalogs justify their maintenance cost when they enable specific workflows that would otherwise require manual investigation. Each workflow has different accuracy requirements, and getting this wrong means either over-engineering (requiring fields nobody uses) or under-engineering (missing data when it matters).

Use Case	Accuracy Needed	Required Fields	Why It Matters
Incident response	98%+	owner, oncall, runbook, dependencies	Wrong owner during incident extends MTTR by 15-30 minutes



Use Case	Accuracy Needed	Required Fields	Why It Matters
Security scanning	95%	repository, runtime, data classification	Missing services mean unscanned attack surface
Cost attribution	90%	owner, cost center, environment	10% error in cloud bills is acceptable for most orgs
Dependency analysis	85%	dependencies, consumers, API endpoints	Used for planning, not emergencies
Developer onboarding	80%	description, documentation, owner	Stale docs annoy but don't cause outages

Accuracy requirements by use case.

The accuracy thresholds aren't arbitrary. Incident response has the highest bar because getting it wrong has immediate, measurable consequences – every minute spent paging the wrong team is a minute your users are affected. Security scanning needs high accuracy because you can't secure what you don't know exists. Cost attribution can tolerate more noise because finance teams already expect some allocation disputes.

This framing helps with prioritization. If your primary use case is incident routing, you can defer the cost center fields until you've nailed ownership. If you're trying to pass a SOC 2 audit, data classification¹ becomes required immediately.

Catalog vs CMDB vs Service Mesh

Service catalogs often get confused with CMDBs (Configuration Management Databases) or service mesh observability. They serve different purposes and have different data models.

A CMDB (Configuration Management Database) is an IT operations tool, typically focused on hardware and infrastructure components with manual entry and periodic audits. CMDBs (Configuration Management Databases) excel at tracking physical assets and their relationships but struggle with the pace of change in microservices environments. A service catalog is developer-facing: it tracks logical services, their ownership, and how they relate to each other.



Service mesh observability (from tools like Istio or Linkerd) provides real-time traffic data but doesn't know about ownership, business context, or services that aren't currently receiving traffic. It tells you *what's calling what* right now, not *who's responsible* or *what should be calling what*.

Aspect	Service Catalog	CMDB	Service Mesh
Primary audience	Developers, SREs	IT Operations	Runtime systems
Data source	Declarative + discovered	Manual entry	Runtime observation
Update frequency	On deploy/change	Periodic audits	Real-time
Ownership tracking	Primary purpose	Often missing	Not applicable
Dependency data	Declared + inferred	Declared only	Observed only
Accuracy model	Continuous validation	Point-in-time	Always current (for running services)

Catalog vs CMDB vs service mesh.

The ideal setup uses all three: the catalog declares intent and ownership, the service mesh observes actual traffic, and the CMDB (Configuration Management Database) tracks underlying infrastructure. Drift between catalog declarations and mesh observations surfaces undeclared dependencies or stale entries.

Schema Design Principles

Schema design is where most catalog initiatives go wrong. Teams either start with a sprawling schema that nobody wants to fill out, or they start too minimal and realize six months later they're missing critical fields. The goal is a schema that's small enough to achieve high adoption but complete enough to answer the questions you actually need answered.

Core Entity Model

A service catalog schema centers on the **service** as the primary entity, with relationships to teams, other services, repositories, and runtime environments. The relationships matter as much as the entities themselves.



#	Entity	Key relationships
1	Service	Owned by Team, depends on Service, sourced from Repository, belongs to Domain
2	Team	Owns Service, has Person members
3	Repository	Source of Service definition
4	Domain	Business grouping for Service ownership and reporting

Core catalog entities and relationships.

The service entity needs several categories of fields:

#	Category	Fields	Purpose
1	Identity	id, name, slug	Establish uniqueness
2	Classification	tier, lifecycle, domain, type	Determine automation behavior
3	Ownership	owner, contacts	Enable incident routing
4	Technical	repository, language, runtime	Support security scanning and dependency analysis
5	Operational	runbook, documentation, SLO reference	Support incident response

Service entity field categories.

Here's a concrete example using Backstage's `catalog-info.yaml` format, which has become a de facto standard:



catalog-info.yaml

```
1  # Backstage service catalog entry
2  apiVersion: backstage.io/v1alpha1
3  kind: Component
4  metadata:
5    name: payment-processor
6    description: Processes credit card transactions
7    annotations:
8      github.com/project-slug: acme/payment-processor
9      pagerduty.com/service-id: PABC123
10     datadoghq.com/dashboard-url: https://app.datadoghq.com/dashboard/abc-123
11   tags:
12     - payments
13     - pci-scope
14  spec:
15    type: service
16    lifecycle: production
17    owner: team-payments
18    system: checkout
19    dependsOn:
20      - component:user-service
21      - resource:payments-postgres
22    providesApis:
23      - payment-api
```

Backstage catalog entry with ownership, dependencies, and tool integrations.

Required vs Optional Fields

The biggest adoption killer is requiring too many fields upfront. I've seen catalogs launch with 30+ required fields – teams take one look and decide they'll "get to it later." Start with the absolute minimum that enables your primary use case.

For incident routing (the most common primary use case), you need exactly five required fields:

owner:		Which team is responsible
tier:		How critical is this service (determines incident priority)



repository:	Where's the code
oncall_schedule:	Who gets paged (can be on owner team)
lifecycle:	Is this in production, deprecated, etc.

Everything else can be recommended or optional at launch. Once you hit 90%+ coverage on those five fields, you can start requiring additional fields like runbooks for tier-1 services or dependency declarations.

Field	Requirement Level	Enforcement Point	Rationale
owner	Required	CI + deploy gate	Can't route incidents without it
tier	Required	CI	Determines SLO requirements
repository	Required	CI	Source of truth for service
lifecycle	Required	CI	Prevents paging for deprecated services
dependencies	Required for tier-1/tier-2	Periodic audit	Impact analysis for critical services
runbook	Required for tier-1/tier-2	Deploy gate	tier-1 incidents need documented response
documentation	Recommended	Periodic audit	Improves discoverability
slo	Recommended	Periodic audit	Best practice, not blocking

Field requirements with enforcement points.



INFO

Launch with 5-7 required fields that enable incident routing. Add requirements as catalog coverage improves. A catalog with 95% coverage on five fields beats one with 60% coverage on twenty fields.

Extension Points and Custom Metadata

Different teams have different metadata needs. The security team wants data classification and compliance scope. FinOps wants cost centers and budget owners. Platform teams want runtime details. Rather than cramming everything into the core schema, design an extension mechanism.

The pattern I've seen work best uses namespaces extensions with their own validation schemas. Each extension has an owner (usually a platform team like security or FinOps), defines which services it applies to, and specifies whether it's required or optional.

catalog-info.yaml

```
1  # Service with compliance and finops extensions
2  apiVersion: backstage.io/v1alpha1
3  kind: Component
4  metadata:
5    name: payment-processor
6    annotations:
7      # Compliance extension (required for production services)
8      compliance.acme.com/data-classification: restricted
9      compliance.acme.com/pci-scope: "true"
10     compliance.acme.com/pii-handled: "true"
11     compliance.acme.com/last-audit: "2024-11-15"
12
13     # FinOps extension (optional)
14     finops.acme.com/cost-center: CC-4521
15     finops.acme.com/budget-owner: jane.smith
16  spec:
17    type: service
18    lifecycle: production
19    owner: team-payments
```



Namespace extensions via annotations in Backstage.

The key is that core catalog fields stay stable while extensions can evolve independently. The security team can add new compliance fields without coordinating with every service owner. Extensions can have their own validation rules – the compliance extension might require `data-classification` for all production services, while the FinOps extension remains fully optional.

This separation also helps with adoption. Teams can satisfy the core requirements quickly, then fill in extensions as they have time or as specific compliance deadlines approach.

i INFO

Version your extension schemas. When the security team adds a new required field, give teams a migration window rather than breaking all CI pipelines at once.

Ownership Model Design

If I had to pick one field that determines whether a catalog succeeds or fails, it's ownership. Every other use case – incident routing, cost attribution, security scanning – depends on knowing who's responsible. Get ownership wrong and you've built an expensive spreadsheet.

But “ownership” is deceptively simple. A service might have a development team that writes the code, an SRE team that handles production incidents, a security contact for vulnerability disclosures, and a cost owner for budget decisions. Flattening all of that into a single `owner` field creates ambiguity during incidents.

Ownership Hierarchy

The ownership model needs to distinguish between different types of responsibility. I've found that a two-tier approach works well: a **primary owner** who's responsible for the service's existence and development, plus **role-specific contacts** for specialized functions.



#	Role	Responsibility	When they're contacted
1	Primary owner	Service development, roadmap, architecture decisions	Default for anything not covered by a specific role
2	Oncall	Incident response, production issues	Automated alerts, P1/P2 incidents
3	Security contact	Vulnerability remediation, security reviews	CVE notifications, penetration test findings
4	Change approver	Production deployment approval	Release management workflows
5	Cost owner	Budget decisions, optimization	FinOps alerts, capacity planning

Ownership roles and their triggers.

The primary owner is usually the team that writes the code. They're accountable for the service's health, performance, and continued development. Role-specific contacts handle specialized functions – the security contact might be a designated engineer on the team, or it might be a security team member embedded with the product organization.

Here's how this looks in a Backstage catalog entry:

catalog-info.yaml

```

1  # Service ownership with role-specific contacts
2  apiVersion: backstage.io/v1alpha1
3  kind: Component
4  metadata:
5    name: payment-processor
6    annotations:
7      pagerduty.com/service-id: P123ABC
8      opsgenie.com/team: payments-oncall
9  spec:
10   type: service
11   lifecycle: production
12   owner: team-payments
13

```



```

14      # Role-specific contacts via custom extension
15      x-contacts:
16          oncall: payments-oncall-schedule
17          security: alice.chen
18          change-approver: team-payments-leads
19          cost-owner: payments-engineering-manager

```

Backstage entry with primary owner and role-specific contacts.

For most services, the primary owner handles all roles. You only need role-specific contacts when there's a reason to route differently – a shared SRE oncall rotation, a dedicated security champion, or a manager who approves production changes.

Ownership Transfer Workflows

Services change hands. Teams get reorganized, people leave, priorities shift. Without a formal transfer process, ownership data rots – the catalog still says Team A owns the service, but Team A disbanded six months ago.

The transfer workflow needs to ensure that:

- The current owner explicitly releases responsibility
- The new owner explicitly accepts it
- Critical knowledge transfers before the handoff completes
- Both teams are on the escalation path during transition

A two-week transition period where both teams receive pages has saved me more than once. The new team gets exposure to real incidents while the old team is still available to help. It surfaces knowledge gaps before they become 3 AM surprises.

Transfer phase	Duration	Who's on-call	What happens
Pending approval	Until both approve	Old team only	Transfer request sits in queue



Transfer phase	Duration	Who's on-call	What happens
Knowledge transfer	1 week typical	Old team only	Runbook review, access provisioning, documentation walkthrough
Transition period	2 weeks default	Both teams	New team primary, old team escalation
Complete	---	New team only	Old team removed, catalog updated

Ownership transfer phases.

The knowledge transfer checklist should cover:

#	Item	Details	Automatable?
1	Runbook review	Walk through incident procedures	No
2	Access provisioning	AWS, Kubernetes, CI/CD, monitoring	Yes (via IDP)
3	Oncall schedule	Update PagerDuty/Opsgenie rotation	Yes (via API)
4	Documentation review	Answer questions, update gaps	No
5	Stakeholder notification	Dependent service owners, PMs	Yes (via catalog)
6	Cost center update	FinOps allocation change	Yes (via API)

Knowledge transfer checklist.

Automate as much of this as possible. The transfer request can trigger access provisioning workflows, update the oncall schedule via API, and notify stakeholders automatically. Manual steps are where transfers stall.



Orphan Detection and Resolution

Services become orphaned when their owning team dissolves, empties out, or goes inactive. This happens more often than you'd think – reorgs, layoffs, and attrition all create orphans. A tier-1 service with no valid owner is a ticking time bomb.

Run orphan detection weekly. The rules should catch:

Dissolved teams:	The team ID in the catalog no longer exists in your identity provider
Empty teams:	The team exists but has zero members
Inactive teams:	The team hasn't deployed, committed, or responded to incidents in 90+ days
Missing oncall:	tier-1 / tier-2 services without a valid oncall schedule

orphan-detection-config.yaml

```

1  # AWS EventBridge rule for weekly orphan detection
2  Resources:
3    OrphanDetectionRule:
4      Type: AWS::Events::Rule
5      Properties:
6        Name: catalog-orphan-detection
7        Description: Weekly check for orphaned services
8        ScheduleExpression: "cron(0 9 ? * MON *)"
9        State: ENABLED
10       Targets:
11         - Id: orphan-detection-lambda
12           # Lambda queries catalog API and identity provider to find
13           # services with dissolved teams, empty teams, or missing oncall
14           Arn: !GetAtt OrphanDetectionLambda.Arn
15           Input: |
16             {
17               "rules": ["dissolved-team", "empty-team", "inactive-team", "missing-
oncall"],
18               "notify": ["platform-team-slack", "engineering-directors"],

```



```
19         "autoEscalate": {  
20             "tier-1": "domain-owner",  
21             "tier-2": "domain-owner"  
22         }  
23     }
```

AWS EventBridge rule for scheduled orphan detection.

For critical services, consider auto-escalation. When a tier-1 service is orphaned, automatically assign it to the domain owner or a catch-all platform team until proper ownership is established. This ensures someone gets paged even when the original team no longer exists.

DANGER

A tier-1 service without a valid owner means the next incident has no one to page. Treat orphan detection failures for critical services as a P1 issue.

With ownership properly tracked and enforced, you have the foundation for the second most valuable catalog data: dependencies.

Dependency Tracking

Dependencies are the second most valuable data in a service catalog, after ownership. Knowing what depends on what enables impact analysis before maintenance, blast radius calculation during incidents, and migration planning for deprecations. But dependency data has a shelf life – services add new dependencies constantly, and declared dependencies go stale when code changes but the catalog doesn't.

Dependency Types and Metadata

Not all dependencies are equal. A service that can't function without its database has a *critical* dependency. A service that falls back to cached data when a recommendation engine is down has an *optional* dependency. Capturing this distinction matters for incident response and maintenance planning.



#	Dependency type	Description	Example
1	Runtime critical	Service fails if dependency is unavailable	Database, auth service
2	Runtime degraded	Service works with reduced functionality	Recommendation engine, analytics
3	Startup only	Needed at boot but not ongoing	Config service, secrets manager
4	Async/background	Used for non-blocking operations	Message queue, batch processor
5	Development only	Not used in production	Mock services, test fixtures

Dependency types by criticality.

Beyond type, you want to capture the **failure mode**: what happens when this dependency is unavailable? Does the service fail hard, degrade gracefully, retry with backoff, or serve cached responses? This information is invaluable during incidents – it tells you whether a downstream outage will cascade or be contained.

Here's how to declare dependencies in a Backstage catalog entry:

catalog-info.yaml

```

1  # Service dependencies with criticality metadata
2  apiVersion: backstage.io/v1alpha1
3  kind: Component
4  metadata:
5    name: checkout-service
6    annotations:
7      # Dependency metadata via custom annotations
8      deps.acme.com/user-service: "critical, sync, grpc"
9      deps.acme.com/inventory-service: "critical, sync, http"
10     deps.acme.com/recommendations: "optional, sync, http"
11     deps.acme.com/analytics-queue: "optional, async, sqs"
12  spec:
13    type: service
14    lifecycle: production
15    owner: team-checkout
16    dependsOn:

```



```

17      - component:user-service
18      - component:inventory-service
19      - component:recommendations
20      - resource:analytics-queue
21      - resource:checkout-postgres

```

Backstage dependencies with criticality annotations.

Automated Dependency Discovery

Declared dependencies are only half the picture. Services add dependencies all the time – a new API call here, a database connection there – and developers don’t always remember to update the catalog. Runtime observation catches what declarations miss.

The best approach combines multiple data sources:

Source	What it catches	Confidence	Limitations
Catalog declarations	Intentional dependencies	High (if maintained)	Goes stale without enforcement
Service mesh (Istio/Linkerd)	HTTP/gRPC traffic between services	High	Misses non-mesh traffic
Distributed tracing ^[2]	Cross-service calls in traced requests	Medium-high	Sampling may miss infrequent calls
DNS query logs	Service discovery lookups	Medium	Can't distinguish used from resolved
Database connection logs	Direct database dependencies	High	Requires database-level logging

Dependency discovery sources and their tradeoffs.

The reconciliation logic compares declared dependencies against observed traffic. Four outcomes are possible:

Validated: | Declared and observed-this dependency is real and documented



Declared but not observed:	In the catalog but no recent traffic-possibly stale or only used in specific scenarios
Undeclared:	Observed in traffic but not in catalog-shadow dependency, needs to be declared
Unknown:	Neither declared nor observed-shouldn't exist in the graph

Undeclared dependencies are the dangerous ones. They represent hidden coupling that won't show up in impact analysis. When you're planning maintenance on a service, you need to know *everything* that depends on it, not just what's documented.

prometheus-dependency-alert.yaml

```

1  # Alert on undeclared dependencies detected via service mesh
2  groups:
3    - name: catalog-dependency-drift
4      rules:
5        - alert: UndeclaredDependencyDetected
6          expr: |
7            catalog_dependency_status{status="undeclared"} > 0
8          for: 1h
9          labels:
10             severity: warning
11             annotations:
12               summary: "Undeclared dependency from {{ $labels.source }} to {{ $labels.target
13               }}"
14               description: "Traffic observed but dependency not declared in catalog. Update
15               catalog-info.yaml."
16             runbook: https://runbooks.acme.com/undeclared-dependency

```

Prometheus alert for undeclared dependencies.

Dependency Graph Queries

Once you have dependency data, the catalog needs to answer questions about it. The most common queries:

- What does service X depend on? (direct and transitive)



- What depends on service X? (consumers/reverse dependencies)
- What's the blast radius if service X fails?
- Are there circular dependencies?
- What's the critical path between service A and service B?

Blast radius is particularly useful for incident response and maintenance planning. When a tier-1 database goes down, you want to immediately know how many services are affected and which teams need to be notified.

Query	Use case	Output
<code>getDependencies(serviceId, depth=2)</code>	Pre-deployment impact check	Tree of downstream services
<code>getConsumers(serviceId)</code>	Maintenance notification list	List of teams to notify
<code>getBlastRadius(serviceId)</code>	Incident severity assessment	Affected services by tier
<code>detectCycles()</code>	Architecture health check	List of circular dependency chains
<code>findCriticalPath(from, to)</code>	Migration planning	Shortest dependency chain

Common dependency graph queries.

For blast radius specifically, group results by tier. If a service failure affects three tier-1 services, that's a critical incident regardless of how many tier-4 services are also impacted. The tier breakdown helps incident commanders prioritize communication and recovery efforts.

INFO

Include blast radius queries in your service runbooks. When an incident starts, the on-call engineer should immediately know which other services and teams are affected.



Automation and Enforcement

The catalogs that survive are the ones that don't rely on human memory. If updating the catalog is a manual step that happens after deployment, it won't happen consistently. The only way to maintain accuracy is to make the catalog part of the deployment path – validate entries in CI, block deploys when critical data is missing, and detect drift automatically.

Catalog-as-Code Pattern

Store service definitions in the same repository as the service code. This gives you version control, code review, and the ability to enforce changes through CI/CD. When the catalog entry lives next to the code, updating it becomes part of the normal development workflow rather than a separate chore.

The Backstage `catalog-info.yaml` pattern works well here. Each service repository contains its own catalog entry, and a central catalog aggregates entries from all repositories. Changes to catalog metadata go through the same PR process as code changes.

catalog-info.yaml

```
1  # Lives in the service repository root
2  apiVersion: backstage.io/v1alpha1
3  kind: Component
4  metadata:
5    name: payment-processor
6    description: Processes credit card transactions for checkout
7    annotations:
8      github.com/project-slug: acme/payment-processor
9      pagerduty.com/service-id: PABC123
10 spec:
11   type: service
12   lifecycle: production
13   owner: team-payments
14   system: checkout
15
16   # Dependencies declared alongside code
17   dependsOn:
18     - component:user-service
19     - component:fraud-detection
20     - resource:payments-postgres
```



```
21      - resource:kafka-payments
```

Catalog entry in the service repository.

The benefits of catalog-as-code:

Audit trail:	Git history shows who changed what and when
Review process:	Catalog changes get the same scrutiny as code changes
Atomic updates:	Code and catalog changes ship together
Developer ownership:	Teams maintain their own entries rather than relying on a central team

CI/CD Integration

Validate catalog entries on every PR. This catches issues before they reach production: missing required fields, references to teams that don't exist, dependencies on services that aren't in the catalog.

.github/workflows/catalog-validation.yaml

```
1  # GitHub Action for catalog validation
2  name: Catalog Validation
3
4  on:
5    pull_request:
6      paths:
7        - 'catalog-info.yaml'
8
9  jobs:
10   validate:
11     runs-on: ubuntu-latest
12     steps:
13       - uses: actions/checkout@v4
14
15       - name: Validate schema
16         run: |
```



```

17         npx @backstage/cli catalog:validate catalog-info.yaml
18
19     - name: Check owner exists
20       run: |
21         OWNER=$(yq '.spec.owner' catalog-info.yaml)
22         curl -sf "$CATALOG_API/teams/$OWNER" || \
23           (echo "Owner team '$OWNER' not found" && exit 1)
24       env:
25         CATALOG_API: ${vars.CATALOG_API_URL}
26
27     - name: Validate dependencies exist
28       run: |
29         for dep in $(yq '.spec.dependsOn[]' catalog-info.yaml); do
30           curl -sf "$CATALOG_API/entities/$dep" || \
31             (echo "Dependency '$dep' not found" && exit 1)
32         done
33       env:
34         CATALOG_API: ${vars.CATALOG_API_URL}

```

GitHub Action for catalog validation on PRs.

For tier-1 and tier-2 services, make validation failures blocking. For tier-3 and tier-4, warn but allow the PR to merge – you want to reduce friction for less critical services while maintaining strict standards for critical ones.

#	Service tier	Missing owner	Missing runbook	Missing dependencies
1	tier-1	Block deploy	Block deploy	Block deploy
2	tier-2	Block deploy	Block deploy	Warn
3	tier-3	Block deploy	Warn	Warn
4	tier-4	Warn	Optional	Optional

Validation enforcement by tier.



Automated Freshness Enforcement

Even with CI validation, catalog entries go stale. A service's dependencies change, the team reorganizes, the runbook falls out of date. Freshness enforcement catches entries that pass validation but haven't been reviewed recently.

Run freshness checks weekly. The rules should enforce:

Ownership verification:	Every 6 months, require teams to confirm they still own their services
Dependency review:	Every 90 days for tier-1/2 services, flag declared dependencies not observed in traffic
Runbook freshness:	Every 90 days, remind owners to review runbooks for accuracy

prometheus-freshness-alerts.yaml

```

1  # Prometheus alerts for stale catalog data
2  groups:
3    - name: catalog-freshness
4      rules:
5        - alert: CatalogEntryStale
6          expr: |
7            (time() - catalog_entry_last_updated_timestamp) / 86400 > 90
8            and on(service) catalog_service_tier{tier=~"tier-1|tier-2"}
9          for: 24h
10         labels:
11           severity: warning
12         annotations:
13           summary: "Catalog entry for {{ $labels.service }} not updated in 90+ days"
14           runbook: https://runbooks.acme.com/catalog-freshness
15
16        - alert: OwnershipNotVerified
17          expr: |
18            (time() - catalog_ownership_verified_timestamp) / 86400 > 180
19          for: 24h
20          labels:
21            severity: warning

```




```
22     annotations:  
23     summary: "Ownership of {{ $labels.service }} not verified in 180+ days"
```

Prometheus alerts for stale catalog entries.

The verification workflow should be low-friction. Send a Slack message to the owning team's channel with a link to confirm ownership. If no response in a week, escalate to the team's manager. If still no response, flag the service as potentially orphaned.

Drift Detection

Drift detection compares catalog declarations against actual system state. It catches discrepancies that validation can't: a service that exists in Kubernetes but not in the catalog, a team that was deleted from the identity provider, an oncall schedule that was removed from PagerDuty.

Drift type	What it catches	Data sources
Missing service	Running in prod, not in catalog	Kubernetes API, catalog API
Invalid owner	Team ID doesn't exist	Identity provider, catalog API
Invalid oncall	Schedule ID not found	PagerDuty/Opsgenie API, catalog API
Undeclared dependency	Traffic observed, not declared	Service mesh, catalog API
Stale dependency	Declared, no recent traffic	Service mesh, catalog API

Drift detection types and data sources.

Run drift detection daily. For critical drift (invalid owner on tier-1 service, missing oncall), alert immediately. For less critical drift (undeclared dependency on tier-3 service), batch into a weekly report.



⚠ WARNING

Drift detection requires API access to your identity provider, oncall system, and runtime infrastructure. Budget time for integrations when planning your catalog implementation.

Catalog API and Integrations

A catalog that exists in isolation is just a document repository. The catalog becomes valuable when other systems can query it programmatically – incident management systems routing alerts, CI/CD pipelines validating ownership, observability tools enriching telemetry with service metadata. The API design determines how useful the catalog can be to these downstream consumers.

API Design for Catalog Consumers

Backstage provides a catalog API out of the box, but understanding the API surface helps whether you're using Backstage or building something custom. The core operations fall into three categories: entity CRUD, relationship queries, and search.

API category	Operations	Primary consumers
Entity CRUD	Get, list, create, update, delete	CI/CD pipelines, automation scripts
Relationship queries	Dependencies, consumers, ancestry	Incident tools, impact analysis
Search	Full-text, filtered, by owner/tag	Developer portals, discovery tools

Catalog API operation categories.

Backstage exposes a REST API at `/api/catalog/entities` with filtering via query parameters. You can query by kind, namespace, owner, or any annotation:

Bash

```
1 # List all services owned by a specific team
```



```

2  curl "http://backstage:7007/api/catalog/entities?
    filter=kind=component,spec.owner=team:payments"
3  # Returns: [{"metadata":{"name":"payment-api",...},"spec":
    {"owner":"team:payments",...}}]
4
5  # Get a specific entity by its reference
6  curl "http://backstage:7007/api/catalog/entities/by-name/component:default/payment-api"
7  # Returns: {"metadata":{"name":"payment-api",...},"spec":{"...}}

```

Backstage catalog API queries.

For more complex queries – dependency graphs, transitive relationships, impact analysis – GraphQL works better than REST. Backstage supports GraphQL through a plugin, or you can add a GraphQL layer on top of any catalog backend:

GraphQL

```

1  # Query service with its dependencies and their owners
2  query ServiceImpact($id: ID!) {
3    service(id: $id) {
4      name
5      tier
6      owner { name slack }
7      dependencies {
8        target { name tier owner { name } }
9        type
10     }
11    consumers {
12      name
13      tier
14    }
15  }
16 }

```

GraphQL query for impact analysis.



INFO

GraphQL shines for client-driven queries where different consumers need different projections of the same data. Incident tools might need owner contact info; CI pipelines might only need dependency lists. One GraphQL endpoint serves both without custom REST endpoints for each.

Integration Patterns

The highest-value catalog integrations connect to systems that already have operational context: incident management, observability, and deployment pipelines.

PagerDuty event routing uses catalog metadata to direct alerts to the right team. PagerDuty's Event Orchestration can query external APIs, including your catalog, to enrich events before routing:

YAML

```
1  # PagerDuty Event Orchestration rule
2  # Routes based on catalog tier and owner
3  orchestration_path:
4    sets:
5      - id: "catalog-enrichment"
6        rules:
7          - label: "Enrich from catalog"
8            actions:
9              # Call catalog API to get service metadata
10             webhook:
11               url: "https://catalog.internal/api/enrich"
12               # Passes alert payload, expects owner/tier back
13             conditions:
14               - expression: "event.custom_details.service exists"
15             - label: "Route tier-1 critical"
16               actions:
17                 escalation_policy:
18                   id: "P123ABC" # Tier-1 escalation policy
19               conditions:
20                 - expression: "event.custom_details.tier == '1'"
```



```
21      - expression: "event.severity == 'critical'"
```

PagerDuty orchestration with catalog enrichment.

Datadog Service Catalog can sync with your internal catalog through their API. Instead of maintaining ownership in two places, export from your catalog to Datadog nightly:

YAML

```
1  # Datadog Service Definition (generated from internal catalog)
2  # https://docs.datadoghq.com/service_catalog/
3  schema-version: v2.2
4  dd-service: payment-api
5  team: payments
6  contacts:
7    - type: slack
8      contact: "#payments-oncall"
9    - type: email
10      contact: payments-team@company.com
11  links:
12    - name: Runbook
13      type: runbook
14      url: https://wiki.internal/runbooks/payment-api
15    - name: Repository
16      type: repo
17      url: https://github.com/company/payment-api
18  tags:
19    - "tier:1"
20    - "domain:commerce"
21    - "lifecycle:production"
22  integrations:
23    pagerduty:
24      service-url: https://company.pagerduty.com/services/PABC123
```

Datadog service definition synced from catalog.

Grafana dashboard generation uses catalog metadata to create per-service dashboards automatically. With Grafana's provisioning API, you can template dashboards using Jsonnet or Grafonnet that pull service metadata:



Jsonnet

```

1  // Grafonnet template for service dashboard
2  // Generates a dashboard per service using catalog metadata
3  local grafana = import 'grafonnet/grafana.libsonnet';
4  local dashboard = grafana.dashboard;
5  local graphPanel = grafana.graphPanel;
6  local prometheus = grafana.prometheus;
7  local annotation = grafana.annotation;
8
9  // Service metadata injected from catalog at generation time
10 local service = std.extVar('service');
11
12 dashboard.new(
13   title='%s Service Dashboard' % service.name,
14   tags=['auto-generated', 'catalog', service.domain],
15   editable=false,
16   time_from='now-6h',
17 )
18 .addAnnotation(
19   annotation.datasource(
20     name='Deployments',
21     datasource='-- Grafana --',
22     tags=['deploy', service.name],
23     enable=true,
24   )
25 )
26 .addAnnotation(
27   annotation.datasource(
28     name='Incidents',
29     datasource='pagerduty',
30     tags=['incident', service.name],
31     enable=true,
32   )
33 )
34 .addPanel(
35   graphPanel.new(
36     title='Request Rate',
37     datasource='prometheus',
38     span=6,
39   )
40   .addTarget(

```



```
41     prometheus.target(  
42         'rate(http_requests_total{service="%s"}[5m])' % service.name,  
43         legendFormat='{{method}} {{status}}'  
44     )  
45 ),  
46     gridPos={ x: 0, y: 0, w: 12, h: 8 }  
47 )  
48 .addPanel(  
49     graphPanel.new(  
50         title='Error Rate',  
51         datasource='prometheus',  
52         span=6,  
53     )  
54     .addTarget(  
55         prometheus.target(  
56             'rate(http_requests_total{service="%s",status=~"5.."}[5m])' % service.name,  
57             legendFormat='{{method}}'  
58         )  
59     ),  
60     gridPos={ x: 12, y: 0, w: 12, h: 8 }  
61 )
```

Complete Grafonnet template for catalog-driven dashboards.

In practice, each of these integrations solves a different problem and operates on a different cadence. PagerDuty needs real-time accuracy – a stale on-call mapping at 2 AM is worse than no mapping at all. Datadog and Grafana are more forgiving; a daily or on-change sync keeps dashboards and monitors aligned without hammering either API. Terraform tagging is the easiest to get right because it runs in your deploy pipeline, where the catalog data is already available. Slack is the wild card: it's where engineers actually *discover* ownership, so even a simple bot that resolves “who owns this?” queries pays for itself fast. The table below summarizes the trade-offs.

#	Integration	Sync frequency	Primary benefit
1	PagerDuty	Real-time (webhook)	Correct incident routing
2	Datadog	Daily batch	Unified service metadata
3	Grafana	On catalog change	Auto-generated dashboards



#	Integration	Sync frequency	Primary benefit
4	Terraform	Per deploy	Infrastructure tagging
5	Slack	Real-time	Ownership lookups in chat

Common catalog integrations.

Measuring Catalog Health

A catalog without health metrics will silently decay. You'll assume coverage is good because no one's complained, then discover during an incident that half the tier-1 services have stale owner information. Measuring catalog health requires tracking three dimensions: coverage (what percentage is cataloged), accuracy (does the data reflect reality), and freshness (when was it last updated).

Catalog Coverage Metrics

Coverage measures how much of your infrastructure exists in the catalog. Start by defining the denominator — what *should* be cataloged. For services, the Kubernetes API or your deployment system provides ground truth. Compare that against what's actually in the catalog.

YAML

```

1  # Prometheus recording rules for catalog coverage
2  groups:
3    - name: catalog-coverage
4      rules:
5        - record: catalog:coverage_ratio
6          expr: |
7            catalog_services_total
8            / kubernetes_services_total{namespace!~"kube-.*"}
9        - record: catalog:coverage_by_tier
10       expr: |
11         catalog_services_total
12         / on(tier) kubernetes_services_total

```

Prometheus recording rules for coverage metrics.



Beyond simple counts, measure *completeness* – what percentage of cataloged services have all required fields populated. A service that exists in the catalog but lacks an owner isn't really cataloged for incident response purposes.

#	Metric	Formula	Target
1	Coverage ratio	Cataloged / Total	> 95%
2	Owner completeness	With valid owner / Cataloged	100%
3	Oncall completeness (tier-1)	With oncall / tier-1 services	100%
4	Dependency completeness	With dependencies / Cataloged	> 80%
5	Freshness (30-day)	Updated in 30d / Cataloged	> 70%

Core catalog health metrics.

Export these metrics to Prometheus so they're available alongside your other infrastructure metrics:

YAML

```

1  # Prometheus metrics exporter (runs as CronJob)
2  apiVersion: batch/v1
3  kind: CronJob
4  metadata:
5    name: catalog-metrics-exporter
6  spec:
7    schedule: "*/15 * * * *" # Every 15 minutes
8    jobTemplate:
9      spec:
10        template:
11          spec:
12            containers:
13              - name: exporter
14                image: company/catalog-metrics:latest
15                env:
16                  - name: CATALOG_API_URL

```



```

17         value: "http://backstage:7007/api/catalog"
18     - name: PUSHGATEWAY_URL
19       value: "http://prometheus-pushgateway:9091"
20     command:
21     - /bin/sh
22     - -c
23     - |
24       BASE_URL="$CATALOG_API_URL/entities?filter=kind=component"
25
26       # Fetch metrics
27       TOTAL=$(curl -s "$BASE_URL" | jq length)
28       WITH_OWNER=$(curl -s "${BASE_URL},spec.owner" | jq length)
29       TIER1_NO_ONCALL=$(curl -s "${BASE_URL},spec.tier=1" | jq '
30         [.] | select(.spec.oncall == null)] | length
31       ')
32
33       # Push to Prometheus
34       PG_URL="$PUSHGATEWAY_URL/metrics/job/catalog"
35       cat <<EOF | curl --data-binary @- "$PG_URL"
36       catalog_services_total $TOTAL
37       catalog_services_with_owner $WITH_OWNER
38       catalog_tier1_no_oncall $TIER1_NO_ONCALL
39       EOF
40     restartPolicy: OnFailure

```

Kubernetes CronJob for catalog metrics export.

Alerting on Catalog Quality

With metrics in Prometheus, you can alert when catalog health degrades. Treat catalog coverage like any other SLO (Service Level Objective) – set targets and alert on breaches.

YAML

```

1  # Prometheus alerting rules
2  groups:
3  - name: catalog-health
4    interval: 5m
5    rules:
6      # Coverage dropped below threshold

```



```

7      - alert: CatalogCoverageLow
8        expr: |
9          catalog_services_total
10         / on() kubernetes_services_total{namespace!~"kube-.*"}
11         < 0.95
12        for: 1h
13        labels:
14          severity: warning
15        annotations:
16          summary: "Service catalog coverage below 95%"
17          description: >-
18            Only {{ $value | humanizePercentage }} of services are cataloged.
19            Run drift detection to identify missing services.
20
21      # tier-1 service without oncall is critical
22      - alert: Tier1MissingOncall
23        expr: catalog_tier1_no_oncall > 0
24        for: 5m
25        labels:
26          severity: critical
27          team: platform
28        annotations:
29          summary: "tier-1 service without oncall schedule"
30          description: >-
31            {{ $value }} tier-1 services have no oncall schedule configured.
32            These services cannot be properly supported during incidents.
33
34      # Orphan count increasing suggests process breakdown
35      - alert: OrphanServicesIncreasing
36        expr: |
37          increase(catalog_orphaned_services_total[7d]) > 5
38        labels:
39          severity: warning
40        annotations:
41          summary: "Orphaned services increasing"
42          description: >-
43            {{ $value }} new orphaned services in the past week.
44            Review team offboarding process.
45
46      # Staleness indicates catalog abandonment
47      - alert: CatalogStalenessHigh
48        expr: |

```



```
49         catalog_services_stale_90d
50         / catalog_services_total
51         > 0.20
52     for: 24h
53     labels:
54         severity: warning
55     annotations:
56         summary: "More than 20% of catalog entries are stale"
57         description: >-
58             {{ $value | humanizePercentage }} of services haven't been
59             updated in 90 days. Consider automated freshness enforcement.
```

Prometheus alerting rules for catalog health.

⚠ WARNING

Don't set coverage targets at 100% on day one. Start with tier-1 services, get those to 100% coverage and accuracy, then expand. A realistic progression: tier-1 at 100% in month one, tier-2 at 95% in month three, full estate at 90% by month six.

Build a Grafana dashboard that shows catalog health at a glance. Include panels for coverage ratio over time, a list of tier-1 services missing oncall schedules, orphan count trends, and a staleness heatmap by team. The platform team should review it weekly; leadership should see a monthly summary. When catalog health becomes visible, it gets attention.

Conclusion

A service catalog's value comes entirely from the accuracy and freshness of its metadata. Start with a minimal schema focused on your most critical use case – usually incident routing. Name, owner, tier, oncall schedule. That's enough to answer "who do I call when this breaks?" Once you've achieved high coverage with the core fields, expand to dependencies, documentation links, and domain classification.

Ownership is the single most important field. Without knowing who owns a service, you can't route incidents, assign responsibility, or track accountability. Model ownership hierarchically (team → group → org) to survive reorgs without mass updates. Build ownership transfer workflows that enforce handoffs rather than leaving



orphaned services behind. Run orphan detection on a schedule and escalate services without valid owners – they're liabilities during incidents.

Dependencies require both declared sources and runtime observation. Developers declare what they **think** they depend on; service mesh telemetry reveals what actually happens in production. When declared and observed don't match, you've found either a documentation bug or an undiscovered dependency. Both are worth investigating.

Automation makes the catalog self-sustaining. Validate catalog entries in CI before merging. Block deploys when tier-1 services lack oncall schedules. Detect drift between catalog declarations and runtime reality daily. Integrate the catalog with downstream systems – PagerDuty, Datadog, Grafana – so it becomes the single source of truth that other tools consume rather than duplicate.

Finally, measure catalog health obsessively. Coverage, accuracy, and freshness should be tracked and alerted on like any other critical system metric. When you can demonstrate that 98% of tier-1 services have valid owners and oncall schedules, the catalog stops being a documentation project and becomes infrastructure.

-
- ① Data classification is the process of labeling the types of data a service handles (e.g., Public, Internal, Confidential, or Restricted). It serves as a policy-driven tag that tells the platform what security controls must be applied to a service's underlying infrastructure. ↩



Copyright © 2022 Webstack Builders, Inc.

The text, diagrams, and images in this work are licensed under CC BY-NC 4.0

All code samples in this article are licensed under the MIT License. Feel free to use, modify, and distribute them in any project.

<https://www.webstackbuilders.com/articles/service-catalog-metadata-schema-ownership-tracking>

